SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1513

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct
I Siddhi Srivastava P(192521454) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Siddhi

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud.
   Discuss how cloud storage solutions ensure durability and data redundancy.
   How can encryption and access control protect sensitive backup data?
   Explain the cost advantages compared to traditional storage systems.
   Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
   Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

APPLICATION BASED QUESTIONS

Question 1: Auto-Scaling and Load Balancing for Seasonal Traffic Spikes

Scenario: A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

1. Handling Demand with Auto-Scaling and Load Balancing

Consider an online fashion retailer running a festival sale. On a normal day it serves a few thousand visitors, but when the sale opens at midnight, traffic can jump twenty times within minutes. A fixed set of servers would either be too small (causing crashes) or too large (wasting money all year). The cloud solves this with two cooperating mechanisms.

Load balancing places a load balancer in front of the web and application servers. It spreads incoming requests across all healthy servers in several Availability Zones using algorithms such as round-robin or least-connections. It runs health checks and stops sending traffic to a failed server, and it terminates TLS and can keep sessions sticky when needed.

Auto-scaling automatically changes the number of servers in an Auto Scaling Group. Policies are defined on metrics such as average CPU above 60 percent, requests per second or response time. When traffic grows, new servers are launched from a pre-configured machine image and registered with the load balancer (scale out). When traffic falls, extra servers are terminated (scale in). Scheduled scaling can add capacity before the known start of the sale, and predictive scaling learns from previous sales.

Supporting components make the whole system elastic: a CDN serves images and static pages, an in-memory cache absorbs repeated product reads, a message queue buffers orders, and a managed database uses read replicas.

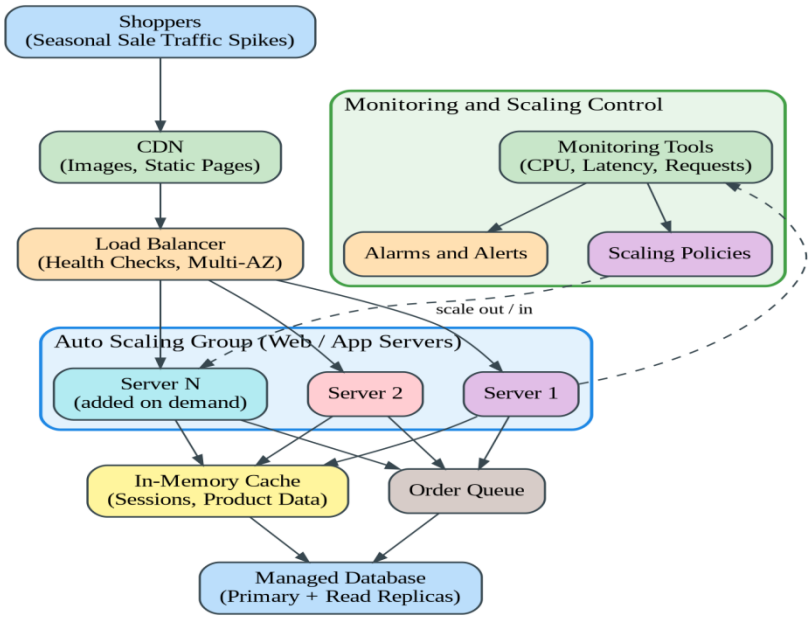


Figure 1: E-Commerce Auto-Scaling and Load Balancing Architecture with Monitoring

2. Reduced Downtime and Improved User Experience

Benefit	How It Is Achieved
Reduced downtime	No single server is critical. Failed servers are replaced automatically, and servers are spread over multiple zones, so a zone failure does not stop the site.
Consistent performance	Capacity grows with demand, so pages load quickly even at peak and customers do not face timeouts at checkout.
Fewer lost sales	Carts and payments keep working during the rush, so conversion and customer trust are protected.
Cost efficiency	The company pays for extra servers only while the sale lasts and scales back afterwards.
Safe deployments	New versions can be rolled out gradually behind the load balancer without taking the site offline.

3. Role of Monitoring Tools

Monitoring tools (Amazon CloudWatch, Azure Monitor, Google Cloud Monitoring, Prometheus with Grafana, Datadog) are the sensing system of the setup. They collect metrics such as CPU, memory, request count, latency, error rate and queue length, as well as logs and traces.

- They **feed the auto-scaling policies**: alarms on thresholds are the trigger that adds or removes servers.
- They **raise alerts** by email, SMS or chat when error rates or latency exceed limits, so engineers can react before customers complain.
- They give **real-time dashboards** during the sale so teams can watch orders, traffic and server health.
- They help **find bottlenecks** such as a slow database query, using logs and distributed tracing.
- They provide **historical data for capacity planning** and cost review after the sale, so the next campaign is tuned better.

Question 2: Secure Cloud Backup Storage

Scenario: An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.

1. Durability and Data Redundancy in Cloud Storage

Cloud object storage services (Amazon S3, Azure Blob Storage, Google Cloud Storage) are designed for extremely high durability, commonly quoted as eleven nines (99.999999999 percent) per year. This is achieved through redundancy at several levels.

- **Multiple copies:** Each object is stored as several copies or erasure-coded fragments on different disks, servers and racks, and normally across at least three Availability Zones.
- **Self-healing:** The service continuously verifies data with checksums and automatically re-creates any copy that is lost or corrupted.
- **Geo-redundancy:** Cross-region replication keeps a copy in a distant region to survive a regional disaster.
- **Versioning and object lock:** Previous versions are retained, and write-once-read-many (WORM) locking prevents deletion or modification, which also protects from ransomware.
- **Service levels:** Providers publish availability and durability commitments in their service level agreements.

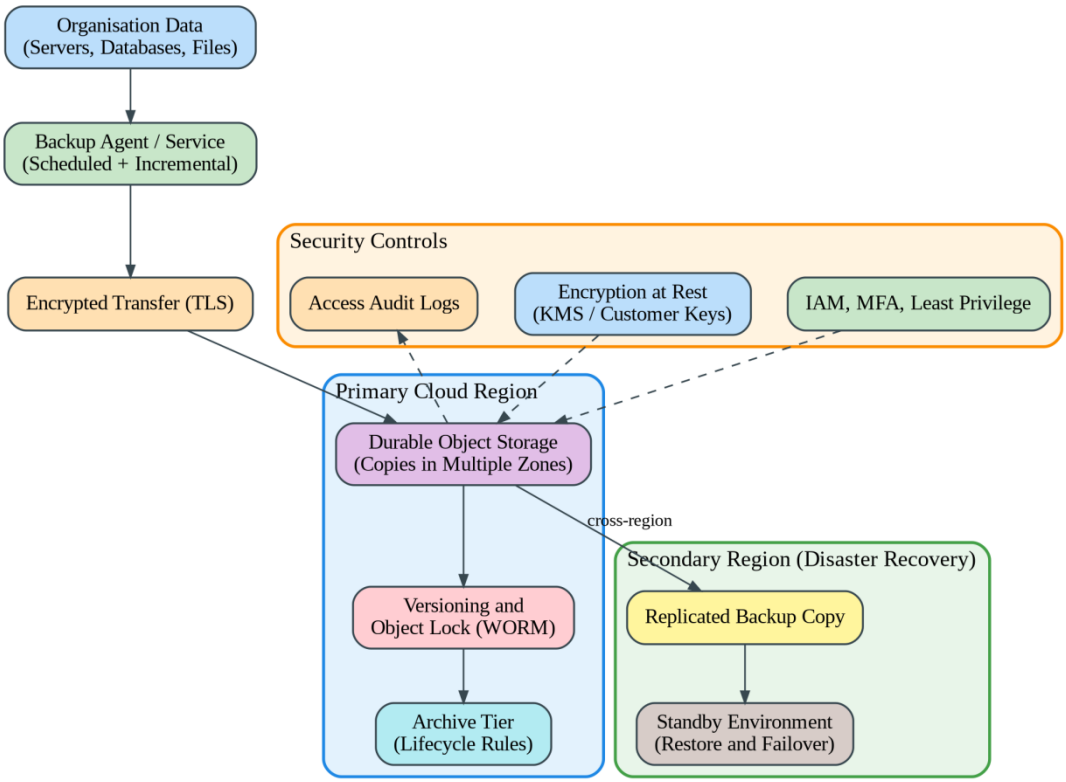


Figure 2: Secure Cloud Backup Architecture with Redundancy, Encryption and Disaster Recovery

2. Protecting Backups with Encryption and Access Control

Encryption: Backup data is encrypted in transit using TLS while it is uploaded, and at rest using AES-256 on the provider side or on the client side before upload. Keys are managed in a Key Management Service or HSM, with customer-managed keys, automatic rotation and the ability to revoke access by disabling a key. Even if someone gained access to the physical storage, the data would be unreadable without the keys.

Access control: Identity and Access Management applies the principle of least privilege, so only the backup service can write, and only a small set of recovery administrators can read or restore. Multi-factor authentication is required for administrators, and MFA Delete or approval workflows protect against accidental or malicious removal. Bucket policies block public

access, private endpoints keep traffic off the internet, and backups are kept in a separate account so that a compromise of the production account cannot erase them. All access is recorded in audit logs and monitored for unusual activity.

3. Cost Advantages Over Traditional Storage

Aspect	Traditional Storage	Cloud Storage
Upfront cost	Large capital expenditure on arrays, tape libraries and data centre space.	No upfront cost; operating expense.
Capacity	Must be bought in advance for future growth, leaving unused capacity.	Pay only for what is stored, scaling without limit.
Operations	Hardware maintenance, replacement, power, cooling and staff.	Managed by the provider.
Redundancy	A second site must be built and operated.	Replication is a setting.
Tiering	Manual movement of data to tape.	Automatic lifecycle rules to cheaper infrequent-access and archive tiers.
Disaster recovery	Costly duplicate infrastructure sitting idle.	Pay for standby storage only, and compute only when needed.

Cost can be reduced further through compression, deduplication and lifecycle policies that expire old backups. Retrieval and data-transfer charges for archive tiers should be considered in the plan.

4. Disaster Recovery Use Case

A hospital group runs its patient management system in a single data centre and takes nightly backups to the cloud. The backups are encrypted, versioned and replicated to a second region. One night a fire damages the data centre. Following the disaster recovery plan, the IT team launches the standby servers in the secondary region using infrastructure templates, restores the latest database backup from cloud storage, repoints DNS to the new environment and verifies the data. Services are back within a few hours (the recovery time objective), and the loss is limited to a few hours of data (the recovery point objective). Because the backup copy was off-site, immutable and encrypted, the organization recovered without the large cost of maintaining a duplicate data centre.

Question 3: Global Deployment Using CDNs and Edge Servers

Scenario: A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

1. How CDNs Improve Performance and Reduce Latency

A Content Delivery Network is a distributed network of servers placed in many cities, called points of presence. Without a CDN, a user in Mumbai opening an app hosted in a US region

has every request travel thousands of kilometres, so latency is high. With a CDN, the request is routed to a nearby server.

- **Caching near users:** Static assets such as images, app updates, videos and configuration files are cached at the edge, so a request is answered locally in a few milliseconds instead of travelling to the origin.
- **Reduced origin load and bandwidth cost:** The origin serves only cache misses, so fewer servers are needed.
- **Optimised connections:** CDNs keep persistent, optimised connections to the origin, use modern protocols (HTTP/2, HTTP/3, TLS session reuse) and compress content.
- **Smart routing:** DNS or anycast routing sends each user to the nearest, healthy edge location.
- **Resilience and security:** Traffic spikes and DDoS attacks are absorbed across the network, and a WAF can filter malicious requests.

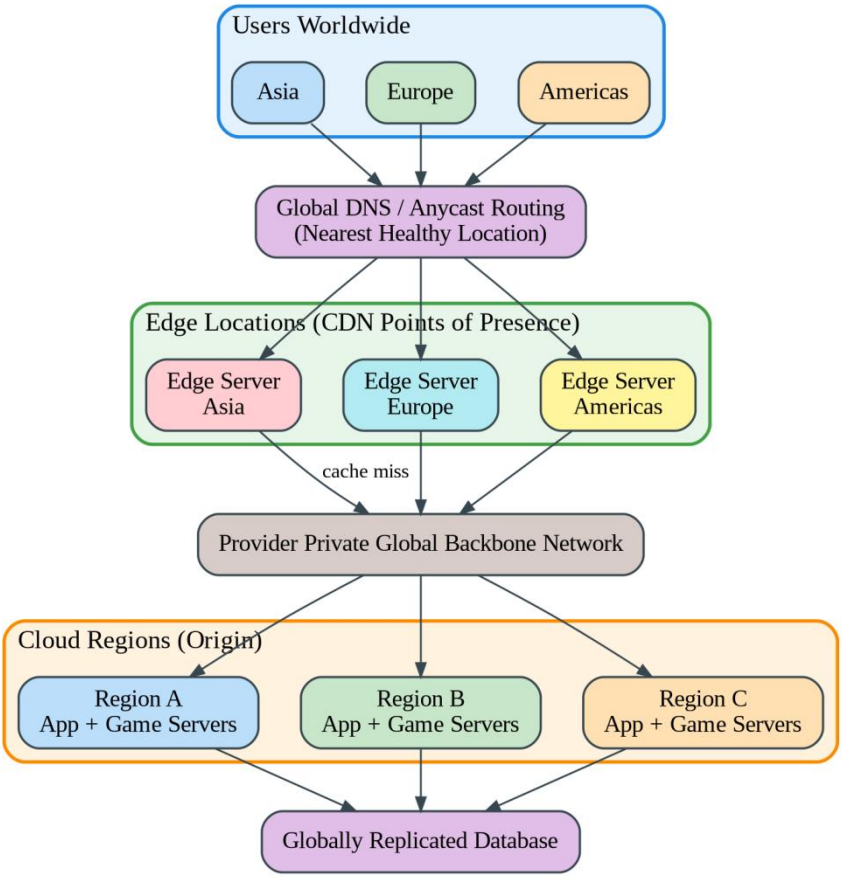


Figure 3: Global Application Delivery Using CDN Edge Servers and Multiple Cloud Regions

2. How Edge Servers Enhance User Experience

- **Lower latency and faster load times:** Content and API responses come from a few milliseconds away, making the app feel responsive.
- **Fewer stalls and failures:** Shorter network paths mean less packet loss and fewer timeouts, especially on mobile networks.

- **Edge compute:** Functions running on edge servers (Lambda@Edge, Cloudflare Workers) can authenticate users, personalise content, resize images and route requests close to the user without a trip to the origin.
- **Consistent experience across regions:** Users in Asia, Europe and the Americas all get similar speed.
- **Local compliance and language:** Edge logic can serve region-specific content and apply local rules.

3. How Cloud Providers Support Global Availability

Capability	Description
Regions and Availability Zones	Dozens of geographic regions, each with several isolated zones, allow the app to be deployed close to users and to survive zone or regional failure.
Global private backbone	High-speed provider network between regions with lower latency and higher reliability than the public internet.
Global load balancing and DNS	Latency-based, geolocation and health-checked routing send users to the best region and fail over automatically.
Global data services	Multi-region replicated databases and storage keep data close to users and available during a failure.
Infrastructure as code and CI/CD	The same stack can be deployed to many regions consistently.
Service level agreements	Published uptime commitments for multi-zone and multi-region designs.

4. Real-Time Streaming and Gaming Example

Streaming: A video streaming app stores its master videos in cloud object storage and encodes them into multiple bitrates. The CDN caches segments of popular videos at edge locations, so a viewer in Chennai receives video from a nearby edge server, with fast start-up and little buffering, while the origin is contacted only for rarely watched content.

Gaming: A multiplayer mobile game deploys game servers in several regions. Players are matched to the nearest region by latency-based routing to keep ping low (ideally under 50 to 100 ms). The CDN delivers game patches and assets, edge servers handle login and authentication, and leaderboards and player profiles are kept in a globally replicated database. If one region fails, players are redirected to the next closest region.

Question 4: Compliance and Data Security for a Financial Institution

Scenario: A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.

1. How Cloud Providers Support Regulatory Compliance and Auditing

Security in the cloud follows the **shared responsibility model**: the provider secures the underlying infrastructure (facilities, hardware, hypervisor and core services), while the institution secures its own data, identities, configurations and applications.

- **Certifications and attestations:** Providers undergo independent audits and publish reports and certificates such as ISO 27001, SOC 1, SOC 2, PCI DSS, and region-specific schemes. Customers can download them through portals (AWS Artifact, Azure Service Trust Portal) to support their own audits.
- **Compliance tooling:** Services such as AWS Config, Azure Policy and Security Command Center check resources against standards and flag non-compliance, and provide benchmark templates.
- **Audit trails:** Every API call and administrative action is recorded (CloudTrail, Azure Activity Log, Cloud Audit Logs).
- **Contracts and controls:** Data processing agreements, right-to-audit clauses, breach notification commitments and regulator-approved architectures for financial services.
- **Governance features:** Organisation-level policies, tagging and separation of environments.

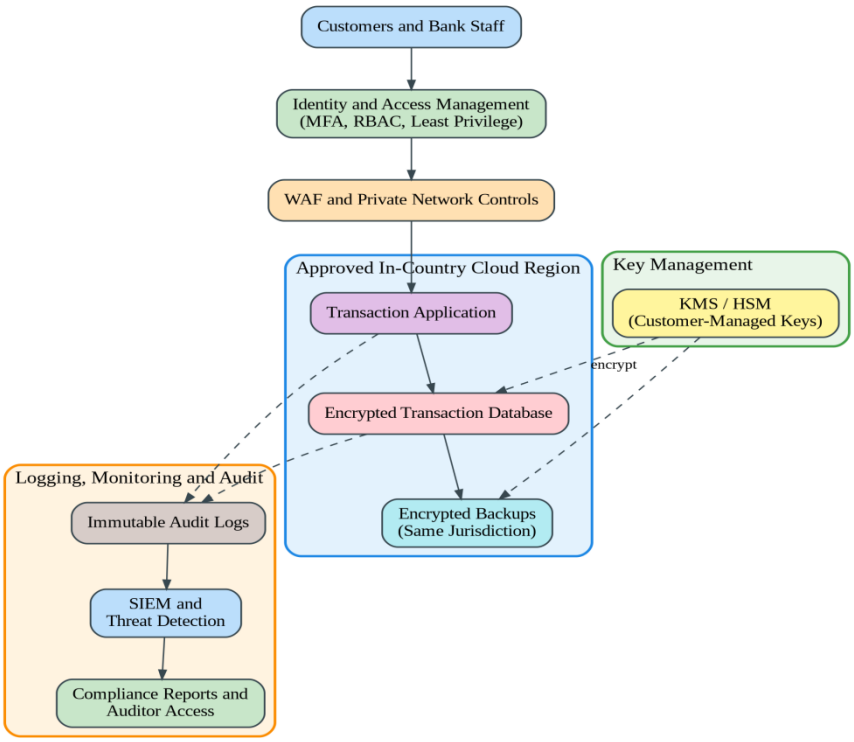


Figure 4: Compliant and Secure Cloud Architecture for Financial Transactions

2. Role of Logging, Monitoring and Encryption in Security

Control	Role in Security
Logging	Creates a tamper-resistant record of who accessed what, from where and when. It supports investigations, evidence for auditors and regulatory reporting. Logs are stored in immutable, access-restricted storage with defined retention.
Monitoring	Continuously analyses logs and metrics in a SIEM, with threat detection and anomaly alerts (for example impossible-travel logins, unusual transfers, privilege changes). Enables fast incident response and proves continuous control.
Encryption in transit	TLS protects data moving between customers, applications and databases, and IPsec protects links to the data centre.
Encryption at rest	AES-256 protects databases, disks, backups and object storage. Keys are held in KMS or HSM with customer-managed keys, rotation, separation of duties and logged key use. Sensitive fields (card numbers, account numbers) can additionally be tokenised.

3. Ensuring Data Sovereignty Requirements Are Met

Data sovereignty means that data is subject to the laws of the country where it is stored or collected, and regulators (for example the Reserve Bank of India) may require financial data to be held within the country. The institution can meet this through the following actions:

- Select cloud regions inside the required country or jurisdiction and use sovereign or government cloud offerings where needed.
- Enforce location restrictions with organisation policies that deny creation of resources in other regions, and keep backups and replicas in approved regions only.
- Disable or control cross-border data transfer, including support access, analytics and logging exports.
- Hold encryption keys in the country, ideally customer-managed or external HSM keys, so that the provider and foreign authorities cannot read the data without the institution.
- Classify data, maintain a data map, and check the contract for sub-processors and data-location commitments.
- Audit regularly using compliance dashboards and evidence of data location.

4. Example Involving Sensitive Financial Transactions

A bank processes online card and UPI payments on a cloud platform. The application runs in a private network within an in-country region. When a customer pays, the request passes through a web application firewall, the customer and staff are authenticated with multi-factor authentication, and the transaction travels over TLS. The card number is tokenised, and the transaction record is stored in a database encrypted with a customer-managed key in an HSM. Every access and change is written to immutable audit logs, and the SIEM raises an alert if an administrator suddenly exports a large number of records. Backups remain in a second in-

country region. During the annual audit, the bank presents the provider's PCI DSS certificate, its own configuration compliance reports and the access logs, demonstrating compliance and sovereignty.

Question 5: Containerization and Kubernetes Orchestration

Scenario: A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

1. Containers versus Virtual Machines

A **virtual machine** includes a full guest operating system running on a hypervisor over virtualised hardware. A **container** packages only the application and its libraries and dependencies, and shares the host operating system kernel with other containers, isolated through namespaces and control groups.

Aspect	**Virtual Machines**	**Containers**
Architecture	Each VM has its own full guest OS on a hypervisor.	Containers share the host OS kernel through a container engine.
Size	Gigabytes.	Megabytes.
Start-up time	Minutes.	Seconds or milliseconds.
Resource usage	High memory and CPU overhead for each guest OS.	Very light, so many more containers fit on one host.
Performance	Slight overhead from virtualisation.	Near-native performance.
Isolation	Strong, hardware-level isolation, suitable for different operating systems and strict multi-tenancy.	Process-level isolation, which is weaker, so extra hardening is needed (a shared kernel vulnerability can affect all).
Portability	Heavier to move between platforms.	Highly portable image runs the same everywhere.
Typical use	Legacy applications, different OS needs, strong isolation.	Microservices, CI/CD, scalable cloud-native applications.

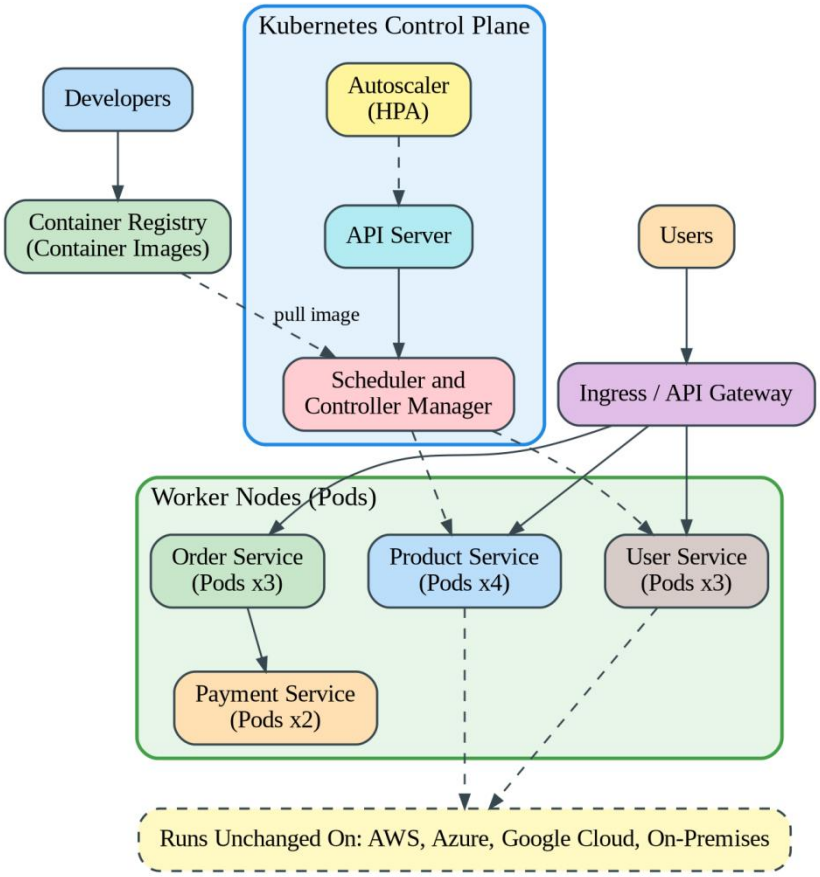


Figure 5: Microservices Application on Kubernetes Running Across Cloud Environments

2. How Orchestration with Kubernetes Simplifies Deployment and Scaling

Running a few containers is easy, but running hundreds across many servers needs automation. Kubernetes is the orchestrator that manages this.

- **Declarative deployment:** The team describes the desired state (image, number of replicas, resources) in YAML files, and Kubernetes makes the cluster match it.
- **Scheduling:** It places containers (pods) on suitable nodes based on resource needs.
- **Self-healing:** Failed containers are restarted, and pods on failed nodes are rescheduled elsewhere.
- **Scaling:** The Horizontal Pod Autoscaler increases or decreases the number of pods based on CPU or custom metrics, and the cluster autoscaler adds nodes.
- **Rolling updates and rollback:** New versions are released gradually with no downtime, and a bad release is rolled back with one command.
- **Service discovery and load balancing:** Services give pods stable names and spread traffic across them.
- **Configuration and secrets:** ConfigMaps and Secrets separate configuration from images.

3. Portability Benefits Across Cloud Environments

A container image bundles the application with everything it needs, so it behaves the same on a developer laptop, a test server, any public cloud or an on-premises data centre. This removes the problem of "it works on my machine". Kubernetes is available as a managed service on all major clouds (Amazon EKS, Azure AKS, Google GKE) and can also run on-premises, giving a common API everywhere.

- Avoids vendor lock-in and makes migration between clouds easier.
- Supports hybrid and multi-cloud deployment using the same manifests and tools.
- Gives consistent environments across development, testing and production, reducing errors.
- Allows the team to choose a provider by cost, region or compliance needs.

Portability is not absolute: dependence on provider-specific services (managed databases, proprietary load balancers) can still reduce it, so teams should use open standards where possible.

4. Example of a Microservices-Based Application

An online food-delivery application is built as independent microservices, each in its own container and deployed as its own Kubernetes deployment:

- **User Service:** registration, login and profiles.
- **Restaurant / Product Service:** menus, prices and availability.
- **Order Service:** cart, order creation and tracking.
- **Payment Service:** payment gateway integration and refunds.
- **Delivery Service:** driver assignment and live location.
- **Notification Service:** SMS, email and push messages.

An ingress or API gateway routes client requests to the right service, and services communicate through REST/gRPC or a message queue. Each service has its own database and can be written in a different language, updated and scaled independently. For example, during the lunch rush the Order and Restaurant services are scaled to many pods while the Payment service keeps only a few. A failure in the Notification Service does not stop customers from placing orders.